Rate Limiting - Cheat Sheet

Rate limiting controls how often a client (or globally, how often anyone) can hit an endpoint. Without it, one client can monopolize the service, retry storms can DDoS your own infrastructure, and runaway loops in client code take down everything they touch.

Done right, rate limits protect the service, the downstream dependencies, and other users. Done wrong, they make legitimate traffic fail unpredictably.

Why Rate Limit

Four main motivations:

- **Protect the service.** Bound the load it can take. Slow clients shouldn't crowd out fast ones.
- **Protect downstream dependencies.** Your service might survive 10x traffic. The DB or the third-party API might not.
- **Fairness.** Prevent one tenant from monopolizing shared resources.
- **Cost control.** Stop runaway usage of paid APIs (OpenAI, SMS providers, mapping services).

Where to Apply Limits

Layer	Limits
CDN / edge	Block obvious abuse (bots, scrapers) before hitting your infrastructure
API gateway	Per-user, per-API-key, per-tenant limits
Application	Business-logic limits (e.g., 5 password resets per hour)
Database	Connection pool limits, per-query rate limits

The closer to the source, the cheaper to enforce. Edge layers are nearly free. In-app limits cost CPU on every request.

Common Algorithms

Each algorithm has different tradeoffs around accuracy, memory cost, and burst handling.

1. Token bucket

A bucket holds N tokens, refilling at a rate of R per second. Each request consumes 1 token. If the bucket is empty, the request is rejected (or queued).

```
capacity = 10 tokens
refill rate = 1 token/sec

Burst: 10 requests instantly, then 1/sec sustained
```

Pros: allows bursts up to the bucket size, smooth sustained rate.

Cons: storing tokens per key requires state.

This is the most common algorithm in practice. AWS API Gateway, Stripe, and Twitter all use variants of it.

2. Leaky bucket

A queue drains at a fixed rate. Requests beyond the queue size are rejected.

```
queue size = 10
drain rate = 1 request/sec
```

Pros: smooth output rate. Good for protecting downstream services that can't handle bursts.

Cons: doesn't allow bursts. Adds latency by queueing.

3. Fixed window counter

Count requests within a fixed time window (e.g., per minute). Reset on the boundary.

```
limit = 100 per minute
12:00:00 – 12:00:59 -> counter
12:01:00 -> reset
```

Pros: trivial to implement (Redis `INCR` + `EXPIRE`).

Cons: edge bursts. 100 requests at 12:00:59 plus 100 at 12:01:00 means 200 in 2 seconds. Not great for tight limits.

4. Sliding window log

Store a timestamp per request. Count entries newer than the window size.

Pros: most accurate.

Cons: memory cost scales with traffic. Bad for high-throughput APIs.

5. Sliding window counter

Approximates the sliding window using two fixed-window counters: current and previous.

```
weight = (time elapsed in current window) / window_size
count = (1 − weight) ∗ previous + current
```

Pros: low memory, smooth, no edge bursts.

Cons: slightly less accurate than the log version.

This is what most production systems actually use.

Per-Key vs Global Limits

Most real systems combine both.

Scope	Use
Per IP	Stop a single client from spamming. Watch for shared NATs.
Per user / API key	The primary axis for SaaS APIs. Different plans get different limits.
Per tenant	Multi-tenant SaaS, isolating noisy customers.
Per endpoint	Tighter limits on expensive endpoints (search, exports).
Global	Hard ceiling on overall traffic regardless of source.

You typically apply several limits in parallel. A request must pass all of them to proceed.

Distributed Rate Limiting

A single instance can keep counters in memory. Multiple instances need shared state, usually Redis.

Redis-based fixed window

```
key: "rate:user:42:minute:202405231230"
INCR key
EXPIRE key 60 (only on first INCR)
if value > limit: reject
```

A Lua script makes it atomic:

```lua
local current = redis.call("INCR", KEYS[1])
if current == 1 then
    redis.call("EXPIRE", KEYS[1], ARGV[2])
end
if current > tonumber(ARGV[1]) then
    return 0
end
return 1
```

Redis-based token bucket

Trickier because tokens refill continuously. Use a Lua script that calculates current tokens based on the last refill timestamp.

For most apps, a library handles this:

- **Bucket4j** (Java): supports in-memory and Redis-backed buckets.
- **Resilience4j RateLimiter**: lighter, in-process only.
- **Redis Cell**: a Redis module that implements the GCRA algorithm.
- **Envoy / nginx**: rate limiting at the proxy layer.

Bucket4j example:

```java
Bucket bucket = Bucket.builder()
    .addLimit(Bandwidth.simple(100, Duration.ofMinutes(1)))
    .build();

if (bucket.tryConsume(1)) {
    // serve the request
} else {
    // 429 Too Many Requests
}
```

HTTP Headers for Rate Limits

When you reject (or even when you allow), tell the client what's happening.

Header	Meaning
`X-RateLimit-Limit`	Total requests allowed in the window
`X-RateLimit-Remaining`	Requests left in the current window
`X-RateLimit-Reset`	Unix timestamp when the window resets
`Retry-After`	Seconds the client should wait before retrying

Example rejection:

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1716468000
Retry-After: 30
Content-Type: application/json

{ "error": "rate_limit_exceeded", "retry_after_seconds": 30 }
```

The IETF standardized `RateLimit` and `RateLimit-Policy` headers (without the `X-` prefix) in 2023. Some APIs use those instead.

Client-Side Handling

When a client hits a rate limit, the right response depends on context.

- **Retry after the `Retry-After` time** with jitter. Don't retry immediately.
- **Exponential backoff** if retry attempts also get limited.
- **Circuit breaker** if rate limits become frequent. Stop hitting the API for a while.
- **Queue and shape traffic** in the client. Some libraries (AWS SDK, Google Cloud client libraries) do this for you.

Never blindly retry on 429. You'll just make the problem worse.

Tools

Tool	Layer	Notes
AWS API Gateway	Edge	Built-in usage plans and rate limits
Cloud Armor / CloudFront	Edge	DDoS protection plus rate limiting
Envoy	Proxy	Local and global rate limiting plugins
nginx	Proxy	`limit_req` and `limit_conn` directives
Kong / Tyk	API gateway	Plugin-based rate limiting
Bucket4j	Java app	Library, in-process or Redis-backed
Resilience4j RateLimiter	Java app	In-process, simple semantics
Spring Cloud Gateway	Java gateway	Built-in `RequestRateLimiter` filter
Redis Cell	Redis module	GCRA algorithm via a Redis command

Best Practices

- **Set limits per axis that matters.** Per-user limits stop one user from monopolizing. Global limits protect the service overall.
- **Use sliding window counter for typical workloads.** Cheap and accurate enough.
- **Return clear 429 responses.** Include `Retry-After` and machine-readable details.
- **Document the limits.** Public APIs need this in the docs.
- **Have a higher limit for premium plans.** Rate limits are also a monetization tool for SaaS APIs.
- **Allow short bursts.** Strict per-second limits frustrate legitimate users. Token bucket handles this well.
- **Test the limits.** Make sure the implementation enforces what the docs claim.
- **Monitor rate limit hits.** A sudden spike in 429s tells you about misbehaving clients or a misconfigured limit.
- **Apply at the edge when possible.** Saves your application from spending cycles on rejected traffic.

Common Gotchas

- **NAT and shared IPs.** Per-IP limits punish entire offices or mobile carriers. Per-account limits are usually better.
- **Inconsistent limits across instances.** Without shared state (Redis), each instance enforces its own count. A 100 req/min limit becomes effectively N*100 with N instances.
- **Race conditions in increments.** A naive `GET, increment, SET` can lose updates under load. Use atomic operations.
- **Clock skew across instances.** Fixed windows depend on synchronized clocks. NTP usually handles this, but be aware.
- **Cost of rate limit checks.** A Redis call per request adds latency. Use local caches for "definitely allowed" cases.
- **Forgetting health checks.** If your health check endpoint hits the same rate limiter, a check storm can mark you unhealthy.
- **Hard limits with no grace.** 429s at exactly 101 requests, when the user paid for 100, feel arbitrary. Allow short bursts.
- **No exemption for internal traffic.** Your monitoring or batch job hits the same limit as a user. Exempt internal traffic with a separate auth signal.
- **Rate limits as security.** Rate limits slow brute force. They don't replace account lockout or proper auth.

Quick Reference

Concept	Key fact
Token bucket	Allows bursts, smooth sustained rate. The common choice.
Leaky bucket	Smooth output, queues bursts
Fixed window	Trivial but edge bursts hurt
Sliding window log	Most accurate, memory-heavy
Sliding window counter	Approximate, low memory. Production default.
Per-user limit	The primary axis for SaaS APIs
Global limit	Hard ceiling on total load
429 Too Many Requests	The standard HTTP status
Retry-After header	Seconds the client should wait
Distributed limit	Needs Redis or similar shared state
Bucket4j	Java library for rate limiting
Edge enforcement	Cheapest, applies before app code runs